

A. Project Overview

Goal: What question are you answering?

- I found which words are commonly used together in hotel reviews, so I could find common themes and ideas in what people write. I also found which words lead to higher or lower reviews.

Dataset: Source, size (link if too large for GitHub).

- Link to data set :
<https://www.kaggle.com/datasets/thedevastator/booking-com-hotel-reviews>
- Kaggle -> Booking.com Hotel Reviews
- Size: 500,000+ rows, 12 columns
- Name in project: booking_reviews copy 2.csv

B. Data Processing

How you loaded it into Rust.

- I used csv and serde crates to deserialize the text and ratings into a Review struct

Any cleaning or transformations applied.

- CLEANING:
 - Converted all text to lowercase
 - Removed punctuation and symbols using regex
 - Tokenized text by white space
 - Removed common English stopwords using stopwords crate

C. Code Structure

Modules and the Purpose of each and rationale for organization

- main.rs -> loading data, graph building, and printing results
- data.rs -> contains review struct and functions to load/clean data
- graph.rs -> defines graph struct and all related graph algorithms
- tests.rs -> tests core functions

Key Functions & Types (Structs, Enums, Traits, etc)

- Review -> struct
 - PURPOSE: stores the data per review

- Load_reviews (path: &str) -> Vec<Review>
 - PURPOSE: reads the CSV and returns analyzed data, also skips rows with invalid ratings
- clean_text(text:&str) -> Vec<String>
 - PURPOSE: converts text to lowercase, strips punctuation, removes stopwords, and tokenizes
- Graph (Struct)
 - PURPOSE: represents undirected co-occurrence graph
- Add_edge (word1, word2)
 - Adds an edge between two words
- add_review(words: &[String])
 - Loops through all word pairs in a review and updates the graph
- degree centrality() -> HashMap<String, usize>
 - Returns number of neighbors for each word
- weighted_degree centrality() -> HashMap<String, usize>
 - Returns sum of edge weights for each word
- pagerank_like_score() -> HashMap<String, f64>
 - Calculates importance of words based on how connected their neighbors are
- find_clusters() -> Vec<Vec<String>>
 - Finds clusters of words using BFS on connected components
- Analyze_review_scores
 - Separates words from high vs low-rated reviews and prints the most frequent ones

Main Workflow

- 1) Load data
- 2) Clean each review
- 3) Build the graph by adding all word pairs
- 4) Compute and print centrality metrics
- 5) Detect clusters
- 6) Compare word usage in high vs. low reviews

D. Tests

cargo test output (paste logs or provide screenshots).

```
running 5 tests
test tests::tests::test_degree centrality ... ok
test tests::tests::test_add_review ... ok
test tests::tests::test_add_edge ... ok
test tests::tests::test_find_clusters_small ... ok
test tests::tests::test_clean_text_basic ... ok

test result: ok. 5 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.02s

crc-dot1x-nat-10-239-81-123:final_project_210_esage jamesage$
```

For each test: what it checks and why it matters

- Test_clean_text_basic
 - Tests that punctuation is removed and words are tokenized
- Test_add_edge
 - Make sure bidirectional edge creation and weight adds on
- Test_add_review
 - Checks that all unique pairs are added
- Test_degree centrality
 - Confirms centrality counts
- Test_find_clusters_small
 - Validates connected components logic

E. Results

All program outputs (screenshots or pasted).

OUTPUT

Graph has 17502 words (nodes)

Top 10 most connected words:

room: 11865 connections

breakfast: 10236 connections

location: 10079 connections

hotel: 9246 connections

good: 9239 connections

staff: 8924 connections

nice: 8487 connections

clean: 8293 connections

great: 7754 connections

friendly: 7379 connections

Top 10 by Degree Centrality

room: 11865

breakfast: 10236
location: 10079
hotel: 9246
good: 9239
staff: 8924
nice: 8487
clean: 8293
great: 7754
friendly: 7379

Top 10 by Weighted Degree Centrality

room: 322098
hotel: 187389
breakfast: 173467
good: 155096
location: 152628
staff: 129964
nice: 125329
clean: 108441
great: 107516
us: 85522

Top 10 by PageRank Score

room: 526.49
hotel: 308.06
breakfast: 302.23
location: 273.68
good: 265.64
staff: 225.32
nice: 216.61
great: 186.00
clean: 177.30
friendly: 141.41

Found 4 clusters.

Top words in HIGH-rated reviews (9-10):

High Rated

available: 4376

comments: 4132
review: 4129
room: 3853
breakfast: 3505
location: 3493
great: 2884
good: 2672
staff: 2643
nice: 2455

Top words in LOW-rated reviews (0-5):

Low Rated
room: 862
hotel: 435
location: 294
available: 279
staff: 251
review: 232
breakfast: 232
comments: 231
bed: 223
good: 221

Interpretation in project context (no need for "groundbreaking" results).

- Words like “room,” “breakfast,” and “location” are central in the graph and were seen a lot in both high and low reviews – this means that they are very important in a hotel experience to visitors
- High-rated reviews had more subjective adjectives like “good,” “great,” and “nice”
- The lower-rated reviews have more concrete “problems” to address like “bed,” “staff,” and “available.”
- Overall, the clustering shows that people value location, cleanliness and service

F. Usage Instructions

How to build and run your code.

- Cargo build

- Cargo run
 - Data is called booking_reviews copy 2
- Takes a while to run -> my computer usually takes abnormally long but around 3+ minutes for me